

GPU Architecture and Security

Course Syllabus

GPU Architecture

CUDA-C

Memory Hierarchy

Side Channels

Security Measurement

Course Information

Course Title:	GPU Architecture and Security
Course Status:	Under-Revised / Draft
Instructor:	Dr. Ahmad Shabani
Course Level:	Advanced Undergraduate / Graduate
Course Type:	Lecture + Programming Assignments + Research-Oriented Project
Recommended Background:	Computer Architecture, Digital Logic Design, C/C++ Programming, Basic Operating Systems
Programming Platform:	CUDA-C/C++, Linux, NVIDIA GPU, Nsight Compute, Nsight Systems, and related profiling tools

Course Description

This course provides a technical introduction to modern GPU architecture with a special focus on security. Students study the GPU hardware execution model, SIMT execution, warp scheduling, streaming multiprocessors, memory hierarchy, CUDA-C programming, performance analysis, and microarchitectural side-channel behavior.

The course connects architectural concepts with practical measurement. Students learn how GPU workloads are mapped to hardware, how memory and execution resources are shared, how performance counters and profiling tools expose low-level behavior, and how shared microarchitectural resources may create security risks in multi-tenant or concurrent execution environments.

Motivation and Relevance

GPUs are now central to high-performance computing, artificial intelligence, scientific simulation, embedded systems, and cloud acceleration. However, their throughput-oriented architecture introduces complex interactions between scheduling, memory hierarchy, caches, interconnects, execution units, and shared resources.

As GPUs are increasingly used in cloud computing, AI inference, edge computing, and security-sensitive workloads, understanding both their performance behavior and their security implications has become essential. This course prepares students to analyze GPUs not only as accelerators, but also as complex microarchitectural systems with measurable and potentially exploitable side effects.

Learning Outcomes

By the end of this course, students should be able to:

- Explain the GPU programming and execution model.
- Describe SIMT execution, warps, thread blocks, grids, and streaming multiprocessors.
- Write, compile, execute, and debug CUDA-C programs.
- Analyze GPU memory spaces including registers, shared memory, local memory, global memory, constant memory, texture memory, L1 cache, and L2 cache.
- Explain memory coalescing, bank conflicts, occupancy, divergence, and latency hiding.
- Use profiling tools such as Nsight Compute and Nsight Systems.
- Interpret GPU performance metrics and relate them to architectural behavior.
- Understand GPU scheduling, contention, and resource sharing.
- Explain microarchitectural side channels and covert channels at a conceptual and experimental level.
- Design controlled experiments to measure timing, cache, memory, and contention effects.
- Evaluate basic mitigation strategies for GPU side-channel leakage.
- Read and present research papers in GPU architecture and security.

Course Modules

Module	Topics
Module 1	GPU evolution, CPU vs GPU design philosophy, throughput-oriented architecture, heterogeneous computing, GPU applications in HPC, AI, cloud, embedded systems, and security-sensitive workloads.
Module 2	CUDA programming model: host/device execution, kernels, grids, blocks, threads, warps, memory allocation, data transfer, synchronization, and error handling.
Module 3	GPU hardware execution model: streaming multiprocessors, SIMT execution, warp scheduling, divergence, occupancy, instruction issue, latency hiding, and execution pipelines.
Module 4	GPU memory subsystem: registers, shared memory, global memory, local memory, constant memory, texture memory, L1/L2 caches, memory coalescing, shared-memory bank conflicts, atomics, and memory consistency concepts.
Module 5	Performance analysis and profiling: CUDA events, timing methodology, Nsight Compute, Nsight Systems, occupancy analysis, roofline-style reasoning, memory throughput, instruction throughput, and performance counters.
Module 6	GPU microarchitectural sharing: caches, memory controllers, interconnects, SM resources, warp schedulers, execution units, DRAM banks, TLBs, and shared contention surfaces.
Module 7	Security fundamentals: threat models, attacker/victim assumptions, side channels, covert channels, timing leakage, contention leakage, cache leakage, memory-system leakage, and isolation challenges.
Module 8	GPU side-channel case studies: timing-based leakage, cache contention, shared-memory effects, memory bandwidth contention, interconnect contention, performance-counter observations, and workload fingerprinting.
Module 9	Defensive and responsible research methods: noise reduction, experiment design, measurement ethics, reproducibility, mitigation strategies, scheduling isolation, partitioning, randomization, and secure GPU usage guidelines.
Module 10	Research project: students design a measurement, profiling, optimization, or security analysis project and present results in a short technical report.

Tentative Weekly Schedule

Week	Main Topic	Description	Activity
1	Introduction to GPU Computing	GPU evolution, CPU vs GPU architecture, throughput computing, heterogeneous systems, GPU applications in AI, HPC, embedded systems, and cloud computing.	Reading
2	CUDA-C Fundamentals	CUDA toolchain, host/device model, kernels, grids, blocks, threads, memory allocation, data movement, and error checking.	Assignment 1
3	GPU Execution Model	SIMT model, warps, thread blocks, streaming multiprocessors, warp scheduling, divergence, occupancy, and latency hiding.	Lab
4	CUDA Programming Patterns	Vector addition, matrix multiplication, reductions, stencil computation, synchronization, atomics, and basic optimization.	Quiz 1
5	GPU Memory Hierarchy I	Registers, local memory, shared memory, global memory, constant memory, texture memory, memory coalescing, and memory bandwidth.	Assignment 2
6	GPU Memory Hierarchy II	L1/L2 cache behavior, shared-memory bank conflicts, DRAM banks, memory latency, memory consistency concepts, and atomic operations.	Lab
7	Profiling and Performance Analysis	CUDA events, timing methodology, Nsight Compute, Nsight Systems, occupancy, memory throughput, instruction throughput, and bottleneck analysis.	Assignment 3
8	Midterm Exam and Review	Review of CUDA programming, execution model, memory hierarchy, and profiling fundamentals.	Midterm
9	Microarchitectural Sharing in GPUs	Shared caches, memory controllers, SM resources, interconnects, warp schedulers, TLBs, DRAM banks, and contention behavior.	Paper Discussion
10	Security Foundations	Threat models, side channels, covert channels, timing leakage, contention leakage, attacker/victim assumptions, and measurement limitations.	Quiz 2
11	GPU Side-Channel Case Studies	Cache-based effects, memory-system contention, timing leakage, interconnect contention, performance-counter observations, and workload fingerprinting.	Assignment 4
12	Experimental Methodology	Noise control, clock effects, warm-up, repetitions, statistical analysis, baselines, reproducibility, ethical boundaries, and responsible disclosure.	Project Proposal
13	Mitigation and Secure GPU Design	Resource partitioning, scheduling isolation, cache and memory isolation, noise injection, randomization, monitoring, and secure deployment practices.	Paper Presentation
14	Advanced Topics	GPU confidential computing, GPU virtualization, MIG-style partitioning concepts, AI workload leakage, multi-GPU systems, and open research directions.	Project Work
15	Final Project Presentations	Student presentations, discussion of results, limitations, future work, and final review.	Presentation
16	Final Exam	Comprehensive final exam.	Final

▣ Assignments and Project Policy

Assignments are designed to connect theory with hands-on GPU experimentation. Students are expected to write CUDA-C programs, profile GPU workloads, analyze performance behavior, and reason about security implications using controlled experiments.

- **Assignment 1: CUDA Basics** — kernel launch, memory allocation, host/device data transfer, error checking, and simple vector operations.
- **Assignment 2: Memory Optimization** — memory coalescing, shared memory tiling, bank conflicts, and performance comparison.
- **Assignment 3: Profiling and Bottleneck Analysis** — use Nsight Compute or Nsight Systems to analyze occupancy, throughput, memory behavior, and execution bottlenecks.
- **Assignment 4: Security Measurement Study** — design a controlled timing or contention experiment and report observations, limitations, and defensive implications.
- **Final Project** — a research-oriented project on GPU architecture, performance, security, profiling, side-channel measurement, or mitigation.

▣ Responsible Security Policy

This course includes security-oriented experiments on GPU microarchitecture. All experiments must be conducted only on authorized systems and only for educational, defensive, and research purposes. Students must not attempt to extract private data, attack third-party systems, bypass access controls, or run unauthorized experiments on shared infrastructure. Any security analysis must be reported with clear assumptions, limitations, and ethical considerations.

▣ Grading Policy

Assessment Component	Weight
Midterm Exam	25%
Final Exam	30%
CUDA Programming Assignments	20%
Security / Profiling Assignment	10%
Final Research Project and Presentation	10%
Class Participation and Paper Discussion	5%

Late Policy: Assignments submitted late receive a 10% penalty per day. Submissions more than one week late may not be accepted unless prior approval is granted. Students must submit source code, a short report, build/run instructions, and representative outputs for each programming assignment.

Final Project

The final project should demonstrate the student's ability to investigate a GPU architecture or security problem using careful methodology. The project may be individual or team-based depending on class size.

Suggested project directions:

- Measuring the effect of memory coalescing and cache behavior on GPU performance.
- Profiling warp divergence and occupancy effects in CUDA kernels.
- Studying shared-memory bank conflicts and their timing impact.
- Evaluating contention between concurrent GPU workloads.
- Characterizing cache or memory-system behavior using microbenchmarks.
- Studying GPU performance-counter behavior under different workloads.
- Surveying and reproducing a safe, simplified version of a GPU side-channel measurement.
- Designing a mitigation or noise-reduction method for timing leakage.
- Comparing security implications of GPU sharing in embedded, desktop, and cloud environments.

Final deliverables:

- Source code and scripts.
- A reproducibility guide.
- A 4–6 page technical report.
- Final presentation.
- Discussion of ethical considerations and limitations.

Required Tools and Platforms

- Linux-based development environment.
- NVIDIA GPU with CUDA support.
- CUDA Toolkit.
- CUDA-C/C++ compiler.
- Nsight Compute.
- Nsight Systems.
- Git and GitHub.
- Python for analysis and plotting.
- Basic shell scripting.
- Optional: NVIDIA Jetson platform, cloud GPU, or multi-GPU system.

Course Materials

This course offers a practical foundation in GPU hardware architecture, CUDA-C programming, memory hierarchy, profiling, and microarchitectural security. It prepares students to analyze GPUs as both high-performance accelerators and shared hardware systems with measurable security-relevant behavior.

Recommended Books and References

Main References

- David B. Kirk and Wen-mei W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, Morgan Kaufmann.
- Shane Cook, *CUDA Programming: A Developer's Guide to Parallel Computing with GPUs*, Morgan Kaufmann.
- John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann.
- NVIDIA, *CUDA C++ Programming Guide*.
- NVIDIA, *CUDA C++ Best Practices Guide*.

GPU Architecture and Parallel Programming

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*.
- Jason Sanders and Edward Kandrot, *CUDA by Example: An Introduction to General-Purpose GPU Programming*.
- Nicholas Wilt, *The CUDA Handbook: A Comprehensive Guide to GPU Programming*.
- Tor M. Aamodt, Wilson W. L. Fung, and Timothy G. Rogers, *General-Purpose Graphics Processor Architecture*.

Security and Research Papers

- Survey papers on microarchitectural side channels and covert channels in heterogeneous systems.
- Research papers on GPU timing channels, cache contention, memory-system leakage, interconnect contention, and workload fingerprinting.
- Recent papers on GPU confidential computing, GPU isolation, AI workload leakage, and secure accelerator deployment.
- Additional research papers will be announced during the semester based on current publications and student project topics.

Prerequisites

Students are expected to have basic knowledge of:

- Computer architecture.
- Digital logic design.
- C or C++ programming.
- Basic operating systems.
- Basic probability and statistics.
- Linux command line.
- Basic parallel programming concepts.
- Basic security concepts.